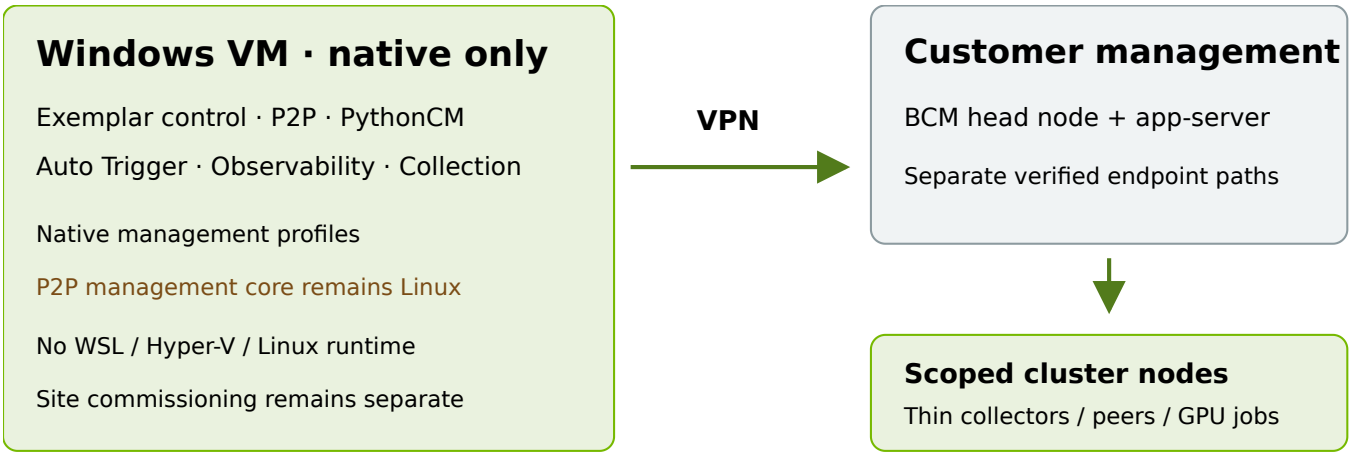
OVA Monorepo Windows

Native Windows deployment and operator guide

Six coordinated tools · Windows preparation · VPN access
Service placement · Offline delivery · Acceptance and recovery



Source baseline: 20261105-windows-native-02 + native profiles
Guide date: 6 November 2026
Read the deployment boundary and validation report before installation.

Contents

From verified source to a measured, recoverable deployment.

1. Six tools and their execution boundaries

2. What the evidence establishes

3. Obtain the complete publication and verify it

4. Prepare Windows and its private workspace

5. Establish VPN connectivity to both customer endpoints

6. Assemble and qualify Rack Status and the P2P console

7. Start the native Observability core

8. Prepare the native Collection Framework

9. Configure one-node Auto Action control

10. Prepare Exemplar recipes and payloads

11. Connect the existing Windows agent and retain evidence

12. Operations, rollback and handover

13. Troubleshooting by boundary

14. Historical delivery and reference material

Native Windows deployment and operator guide

Guide date: 6 November 2026 · Source baseline: `20261105-windows-native-02` plus reviewed native Windows profiles

The management workspace runs natively on an ITSS-provided or approved NVIDIA-internal Windows VM. No WSL, Hyper-V, Linux guest or Linux container runtime is required by the native profiles. GPU tests, Slurm/Enroot/Pyxis and supported thin collectors remain on the customer cluster. The VPN must establish the approved paths from Windows to both the BCM head node and the app-server.

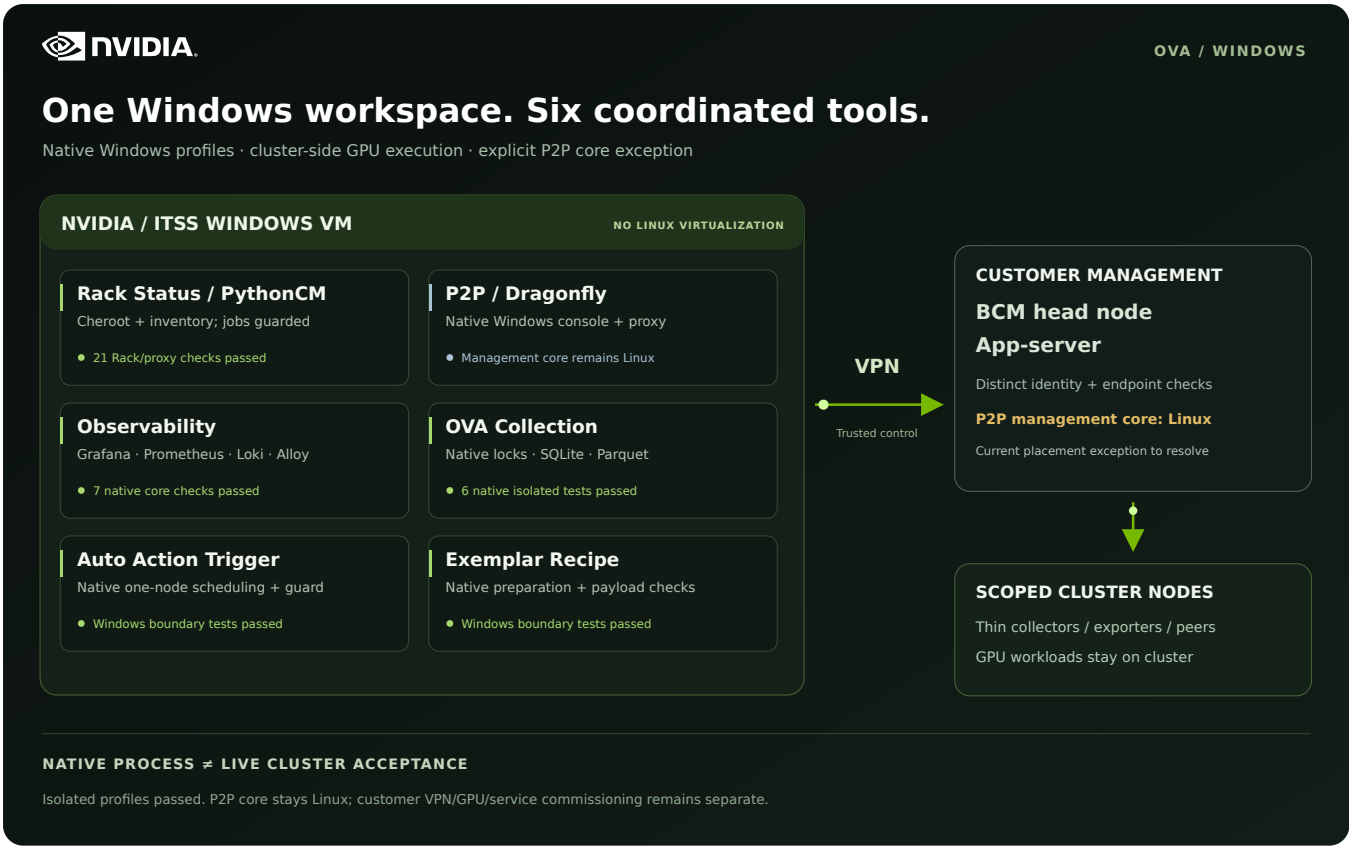
This publication combines the six original toolkits with reviewed Windows adaptations, offline Windows dependencies and explicit acceptance scripts. It is not a claim that six production services are commissioned. **P2P currently has a Windows console/proxy; its Dragonfly management core remains on Linux. PythonCM inventory and PythonCM workload execution have different acceptance boundaries.** Keep both exceptions visible when assessing the central-host requirement.

Use the final receipt index and the native compatibility audit with this guide. Original Linux source findings do not describe the adapted runtime without reconciliation. Isolated Windows acceptance does not prove customer connectivity, complete capture, service-account permissions, production scale or reboot survival.

1. Six tools and their execution boundaries

Component	Native Windows role	Cluster-side role or remaining boundary
Rack Status with PythonCM capability	Adapted Rack backend, matching SPA, Cheroot HTTPS, real Windows locking and ConPTY	Trusted SSH to reviewed BCM helpers; inventory is distinct from workflow execution; PythonCM workflow submission fails closed pending qualification
P2P / Dragonfly	Native static console and Manager API proxy	Manager, scheduler, PostgreSQL, Redis, seeds/peers and origin remain Linux components; central Windows core placement is not delivered
Observability	Grafana, Prometheus, Loki and Alloy as native Windows x64 processes	Thin GPU/node/exporter collectors; approved telemetry route across VPN or an authenticated tunnel
OVA Collection Framework	Derived native collector/processor, SQLite WAL, immutable evidence, Parquet generations, DuckDB and authenticated Waitress browser	Explicit native mirror roots or approved read-only scheduler/source access; optional thin capture hooks
Auto Action Trigger	One-node scheduling, exclusive invocation guard, disabled task definition and durable receipts	Reviewed original launcher and thin node-side scripts; actual GPU tests remain remote
Exemplar Recipe	Pinned preparation, configuration validation, local payload manifests and integrity checks	Official supported recipe, staged image/model, Linux Slurm/Enroot/Pyxis and GPU execution

The requirement is PythonCM capability, rather than a particular Rack repository name. Preserve the existing Windows adaptation of `nvis/nvis-rack-status` and its exported source identity. The older `components/rack-status-pythoncm` snapshot is a comparison/reference source; do not replace the newer adapted implementation merely because the older repository name contains PythonCM.


NVIDIA
OVA / WINDOWS
One Windows workspace. Six coordinated tools.
Native Windows profiles · cluster-side GPU execution · explicit P2P core exception
NVIDIA / ITSS WINDOWS VM
NO LINUX VIRTUALIZATION
Rack Status / PythonCM
Cheroot + inventory; jobs guarded
21 Rack/proxy checks passed
P2P / Dragonfly
Native Windows console + proxy
Management core remains Linux
Observability
Grafana · Prometheus · Loki · Alloy
7 native core checks passed
OVA Collection
Native locks · SQLite · Parquet
6 native isolated tests passed
Auto Action Trigger
Native one-node scheduling + guard
Windows boundary tests passed
Exemplar Recipe
Native preparation + payload checks
Windows boundary tests passed
VPN
Trusted control
CUSTOMER MANAGEMENT
BCM head node
App-server
Distinct identity + endpoint checks
P2P management core: Linux
Current placement exception to resolve
SCOPED CLUSTER NODES
Thin collectors / exporters / peers
GPU workloads stay on cluster
NATIVE PROCESS ≠ LIVE CLUSTER ACCEPTANCE
Isolated profiles passed. P2P core stays Linux; customer VPN/GPU/service commissioning remains separate.

2. What the evidence establishes

Area	Current recorded evidence	Acceptance boundary
Windows control CI	Seven checks passed on Windows Server 2019 / PowerShell 5.1 / native CPython 3.14.10, pipeline `79725803`, job `531758252`, commit `9c6b15938e2430dad78c3e194e92d05873b2581a`	Control parsing, hashes, extraction, six helper tests and nine orchestration tests; excludes customer VPN/GPU/backend commissioning
Actual Windows orchestration	Nine boundary tests passed on the inspected Windows Server 2025 VM with CPython 3.12.10	Native one-node control and Exemplar preparation tests; not a live GPU canary
Native Rack / P2P console	Revision02 official harness passed all 21 native isolated checks with exit 0: fresh offline venvs, dependencies, TLS/WebSocket, ConPTY, real locks, workflow guard and mock Manager proxy; the stability suite ran 46.230 seconds including the full 45-second wait	Configured full UI/auth, customer BCM/Manager and production services remain uncommissioned; no native P2P core claim; metadata-only build ordering comparison is recorded separately
Native Observability	Revision03 passed all seven real native checks on Windows Server 2025 / CPython 3.12.10: readiness, Grafana auth/dashboard/datasources, Alloy metric/log flow and restart persistence	Isolated core acceptance; no customer contact/service installation; Loki required forced-stop fallback, so graceful Loki shutdown remains unqualified
Native Collection	Revision03 passed all six real native tests on Windows Server 2025 / CPython 3.12.10 x64, with no failures/errors/skips; published inputs match the accepted package	Real locking and bounded pointer publication verified; no owned processes left; no customer/Slurm/VPN, service registration or power-loss/reboot qualification
Runtime prerequisites	Actual VM verification passed: Python installer PSF Authenticode valid; both WinSW variants unsigned with expected measured hashes	Verification only; installers/wrappers were not executed or installed; WinSW publisher signing/checksum proof is not claimed

The status table is updated from reviewed receipts before final publication. Consult the exact receipt paths and commit identities in the repository rather than treating a status label as proof. Earlier failures are retained so fixes and retests remain auditable. The passing Windows CI job establishes only its recorded scope.

3. Obtain the complete publication and verify it

Normal Git LFS clone/pull materializes the native Windows dependency closure under `windows/` and the bundled control interpreter. Historical Linux payloads and duplicate top-level archives stay as small LFS pointers by default. Native profile commands need no additional download step. The existing control acceptance and full offline release builder fetch their extra historical inputs automatically using normal Git/LFS credentials. A complete release ZIP already contains its included payloads and works without Git or network access.

```
git clone https://gitlab-master.nvidia.com/nvis-ova/OVA-MONOREPO-WINDOWS.git C:\OVA\repository
Set-Location C:\OVA\repository
git lfs pull
```

Use the organization's normal GitLab credential mechanism. Do not place tokens in clone URLs, scripts, screenshots or logs. If Git/LFS is unavailable on the VM, download the complete offline ZIP linked from the GitLab release on a connected staging host, transfer it through an approved path and verify its checksum and embedded manifest. GitLab-generated source archives are not that offline ZIP.

Directory	Purpose
`windows/`	Primary native profiles and Windows dependencies
`components/`	Pinned original sources and the comparison PythonCM snapshot
`control/`	Sanitized historical coordination delivery; keep its verified tree unchanged
`artifacts/`	Historical control and Linux dependency packs; LFS pointers by default, excluded from the native offline ZIP
`evidence/`	Reviewed validation, intake and sanitization receipts
`docs/`	This guide, compatibility audit and NVIDIA-branded diagram assets

Each native profile verifies its own exact input and dependency hashes before building or starting. Use those verifiers, not a filename as a substitute for byte identity. Preserve upstream license notices and provenance with any redistributed binary closure.

Sanitized historical control artifact

Native profile deployment does not need the duplicate top-level control ZIP. Only for historical helper/toolkit staging, materialize `artifacts/OVA-MONOREPO-WINDOWS-control-sanitized-20261106.zip` with `git lfs pull --include="artifacts/OVA-MONOREPO-WINDOWS-control-sanitized-20261106.zip" --exclude=""`. Read its exact length and SHA-256 from the matching `evidence/sanitized-artifact-provenance.json`. The original r2 ZIP contained generated runtime credential material and is not the

installation/distribution source. Removal paths and derivative hashes are recorded in evidence; original source hashes are historical provenance only.

```
$Publication = 'C:\OVA\repository'
$Provenance = Get-Content -Raw (Join-Path $Publication 'evidence\sanitized-artifact-provenance.json') |
ConvertFrom-Json
$Archive = Join-Path $Publication $Provenance.sanitized_control.path
if ((Get-FileHash -Algorithm SHA256 $Archive).Hash -ne $Provenance.sanitized_control.sha256) { throw 'Archive hash
mismatch' }
if ((Get-Item -LiteralPath $Archive).Length -ne $Provenance.sanitized_control.bytes) { throw 'Archive size
mismatch' }
```

Do not extract payloads into the verified `control/` tree. A retained vendor PostgreSQL image contains Debian's example `ssl-cert-snakeoil.key`; that vendor default is not a site TLS identity. Use freshly issued site-owned keys/certificates for deployed TLS.

4. Prepare Windows and its private workspace

Use the actual Windows account that will operate the tools. Record OS/build, architecture, PowerShell, trusted OpenSSH aliases and available disk. Keep code, runtime state and private evidence separate:

```
C:\OVA\repository\              verified source and dependency closure
C:\OVA-Versions\                newly assembled application versions
C:\OVA-Runtime\                 persistent per-component venv/state/config
C:\OVA-Acceptance\              fresh isolated test instances and receipts
C:\OVA\private\                 site plan and secret file references
```

Never overwrite existing deployed services or state to validate a candidate. New runtime and acceptance paths must be fresh and outside Git. Restrict private directories to their approved identities. In each PowerShell session, set the actual full application interpreter path (replace the placeholder; do not use the embedded 3.14 control interpreter):

```
$Python = '<FULL_APPROVED_PYTHON312_EXE>'
& $Python -c 'import sys, struct; assert sys.version_info[:2] == (3, 12); assert struct.calcsize("P") == 8;
print(sys.version, sys.executable)'
```

Run the native inventory helper for read-only observed host state:

```
Set-Location C:\OVA\repository
.\scripts\Get-NativeWindowsDeploymentEvidence.ps1 `
  -OutputDirectory C:\OVA-Acceptance\inventory-unique `
  -BcmHeadNode <APPROVED_BCM_HOST> -AppServer <APPROVED_APP_SERVER>
```

The helper intentionally omits credential-bearing process/service command lines. Its endpoint probes use the supplied SSH port; prove other application ports separately. Inventory is not functional acceptance.

Use the correct Python runtime

Application wheelhouses target **CPython 3.12 x64**, matching the inspected VM's 3.12.10 environment. The control bundle's embedded CPython 3.14 is for standard-library verification, not the application dependency environment. Use explicit executable paths and component-specific virtual environments.

If a suitable approved Python already exists, reuse it. Otherwise, `windows/runtime/INSTALLATION.md` documents an optional full offline per-user Python 3.12.10 installation. First run its read-only hash and Authenticode verifier:

```
.\windows\runtime\Verify-RuntimeDependencies.ps1 `
  -ReportPath C:\OVA-Acceptance\runtime-signatures-unique.json
```

The verifier executes neither installer nor WinSW. It requires the Python installer signature to be valid with the expected Python Software Foundation organization identity. Resolve trust/revocation failures through ITSS; a measured hash alone does not authenticate a publisher. WinSW's actual signature status is recorded separately, rather than assumed valid. Exact source URLs, licenses and measured hashes are in the runtime dependency manifest.

The optional WinSW `.NET461` variant matches the inspected VM wrapper hash and requires a compatible installed .NET Framework. The standalone x64 alternative is a different binary and needs its own lifecycle qualification. This guide does not register services during source verification or isolated acceptance.

5. Establish VPN connectivity to both customer endpoints

Follow the private Fortinet instructions supplied by the network owner. Use the required client/profile, gateway, certificates and authentication/MFA. Do not invent gateway, routes, DNS or split-tunnel settings. Prestage dependencies if VPN policy blocks NVIDIA GitLab/NVFTP afterward.

Test from the same Windows session/account that runs the applications. Prove BCM and app-server separately:

```
Get-NetIPConfiguration
Get-NetRoute -AddressFamily IPv4
Resolve-DnsName <BCM_HEAD_NODE>
Resolve-DnsName <APP_SERVER>
Test-NetConnection <BCM_HEAD_NODE> -Port <APPROVED_BCM_SSH_PORT>
Test-NetConnection <APP_SERVER> -Port <APPROVED_APP_SERVER_PORT>
ssh.exe -o StrictHostKeyChecking=yes <APPROVED_BCM_ALIAS> hostname
ssh.exe -o StrictHostKeyChecking=yes <APPROVED_APP_SERVER_ALIAS> hostname
```

Replace placeholders using the private site plan. Obtain SSH fingerprints independently; use approved keys/ACLs and `ProxyJump` when required. A TCP success does not prove identity, scheduler rights, collector access or throughput. Keep addresses, route output and credentials outside Git.

Windows-initiated pull/SSH/tunnel paths do not automatically provide a customer-to-Windows callback route. Native Observability binds its receivers to loopback: collectors need an approved authenticated tunnel/gateway or an independently reviewed ingress design. Prove the exact telemetry direction before enrollment; do not turn the VM into a general router or expose unauthenticated receivers broadly.

6. Assemble and qualify Rack Status and the P2P console

The source export from the actual Windows VM is preserved under `windows/rack-p2p/adapters/`. It contains native Rack modifications, PythonCM support/reference helpers and Windows console/proxy code; Linux-only compatibility files are identified separately. The builder verifies the exact packaged base, immutable export, native guard patch, matching static UI and wheelhouse before producing a fresh version. Customer inventory, installed secrets, logs and cache are excluded.

Read `windows/rack-p2p/OPERATIONS.md` and set the actual approved Python path:

```
Set-Location C:\OVA\repository\windows\rack-p2p
$python = Join-Path $env:LOCALAPPDATA 'OVA\Python312\python.exe'
& $python .\build_runtime.py --verify-only
& $python .\build_runtime.py --destination C:\OVA-Versions\rack-p2p-v1
& $python .\acceptance.py --runtime-root C:\OVA-Acceptance\native-rack-p2p-unique
```

The revision02 official isolated acceptance passed all 21 checks with exit 0 on native Windows. It created fresh environments, installed only staged hashed wheels with no package index, checked dependencies and exercised native kernel locks, TLS/WebSocket handling, ConPTY, the PythonCM workflow guard and a loopback mock Manager proxy boundary. The TLS/WebSocket suite lasted 46.230 seconds, including the complete 45-second stability wait. Canonical build-receipt ordering is a separate metadata comparison; this result does not claim a cross-platform receipt-hash match. It does not register services, copy installed VM state, create a real BCM execution session or contact the customer cluster.

The native backend serves through Cheroot; the shared source dependency list's Gunicorn entry is not a Windows serving claim. Real kernel locks and ConPTY preserve concurrency/process behavior rather than bypassing unsupported POSIX calls. For configured startup, inspect the assembled Rack `windows/Start-RackStatus.ps1` and the export's private site configuration contract. Full application authentication, SSH terminal behavior and service-account identity require their own configured acceptance.

PythonCM inventory is not PythonCM workflow submission

The Windows process uses trusted SSH and matching SDK helpers on the BCM head, rather than importing a version-locked SDK locally. Exported evidence distinguishes successful read-only inventory from failed ProgramRunner execution-session-cookie validation. The reviewed workflow also omitted explicit environment, launch-mode and execution-user forwarding.

The assembled derivative therefore rejects PythonCM workflow submission before `submit` with no override switch. This preserves inventory while failing closed on the unqualified execution path. Low-level helper presence does not authorize its use. ProgramRunner fanout does not provide Slurm GPU/GRES/exclusive allocation semantics; Slurm comments are not a PythonCM workload contract. Review the exact patch/provenance and regression evidence before changing this boundary.

P2P placement exception

The Windows console serves the matching static UI and forwards Manager HTTP API requests through an existing trusted SSH tunnel. It does not run Manager, scheduler, PostgreSQL, Redis, seeds or origin natively. Those components retain Linux placement on the designated app-server/cluster infrastructure. A successful mock proxy or console page does not establish real Manager login, seed-to-peer bytes or a native Windows core.

For the historical basic Dragonfly deployment, use the dedicated minimal-profile handoff and keep its patched Manager provenance. Do not invoke Rack's older full-stack P2P provisioning action to overwrite it. NGC, MinIO, models and GPU allocations are not prerequisites for the public basic console. Optional advanced object storage has independent dependencies. Keep the explicit Linux-core exception in the handover until the architecture requirement is resolved.

7. Start the native Observability core

`windows/observability/` stages official Windows x64 Grafana OSS 11.1.4, Prometheus 2.54.1, Loki 3.5.12 and Alloy 1.5.1 archives. They are compatibility pins from the source delivery, not claims about current vendor releases. The profile verifies archives and executable identities before launch; startup performs no downloads.

```
Set-Location C:\OVA\repository\windows\observability
& $Python .\native_core.py verify
.\Test-NativeObservability.ps1 `
  -RuntimeRoot C:\OVA-Acceptance\observability-unique `
  -ReportPath C:\OVA-Acceptance\observability-unique-result.json `
  -PortBase 19000 -PythonExe $Python
```

Acceptance starts fresh native processes, checks readiness/authenticated Grafana, posts a real metric through Alloy into Prometheus, posts a log through Alloy into Loki, queries both, checks the core dashboard/datasources, restarts the same runtime and rechecks retained data. Revision03's actual Windows receipt establishes all seven checks passed, with no owned process/listener left behind. It uses synthetic local telemetry and no customer discovery. Loki required bounded forced-stop fallback in both cycles: restart persistence passed, but graceful Loki shutdown is unqualified.

After isolated acceptance and approved private configuration, start a separate persistent runtime:

```
.\Start-NativeObservability.ps1 -RuntimeRoot C:\OVA-Runtime\observability -PortBase 18000 -PythonExe $Python
# Open http://127.0.0.1:18001 on the Windows VM.
.\Stop-NativeObservability.ps1 -RuntimeRoot C:\OVA-Runtime\observability -PythonExe $Python
```

The wrapper securely prompts for the initial `ova-admin` Grafana password; automated launch may use the documented private environment variable. The secret is not written to source/profile. Existing Grafana credentials are not reset on restart.

Default loopback port	Purpose
`18001`	Grafana browser/authenticated API
`18002`	Prometheus API and remote-write receiver
`18003` / `18004`	Loki HTTP / internal gRPC
`18005`	Alloy readiness/metrics
`18006`	Alloy log receiver `/loki/api/v1/push`
`18007`	Alloy metric receiver `/api/v1/metrics/write`

All bind `127.0.0.1`; no Windows firewall or VPN listener is opened. Prometheus/Loki/Alloy receivers rely on that loopback boundary and require reviewed authenticated ingress for remote collectors.

Discovery starts empty. Add one approved exporter/source and verify timestamps, node/cluster labels and error logs before expansion.

This is a bounded four-service profile with one native-core dashboard. It does not restore/qualify all 106 historical dashboards, Infinity plugin, lake writer/browser or every backend-specific query. Prometheus replaces the original Linux VictoriaMetrics process; preserved datasource UIDs are compatibility aliases, not MetricsQL or VictoriaMetrics API implementations. Local Prometheus retention is two days and Loki retention seven days; production capacity/retention need separate sizing.

Foreground startup is not unattended supervision. `native_core.py serve` is the blocking action for a reviewed service wrapper; `start` returns and is unsuitable as the service executable. Verify process stop/cleanup, account ACLs, restart and reboot behavior before commissioning WinSW/SCM. Bounded process-stop fallback does not prove graceful database shutdown.

8. Prepare the native Collection Framework

`windows/collection/` derives the pinned application into a fresh external runtime with an explicit input-hash receipt. It provides real Windows `LockFileEx` exclusion, SQLite WAL, compressed evidence, Parquet generations, DuckDB and authenticated Waitress browsing. The ten-wheel closure targets CPython 3.12 x64 and installs offline with hashes.

Create private settings from the collection schema: explicit Windows data/source paths, unique cluster/epoch identity, browser loopback port and an external token file of at least 32 characters. Keep credentials outside collected roots. Use `native_scheduler_enabled: false` and disabled remote capture for isolated mirror-mode validation. Read and explicitly acknowledge the native durability boundary before setting `native_windows_durability_acknowledged: true`.

```
Set-Location C:\OVA\repository
.\windows\collection\Prepare-NativeCollection.ps1 `
  -PythonExe $Python `
  -VenvDirectory C:\OVA-Runtime\collection-venv-v1 `
  -RuntimeDirectory C:\OVA-Versions\collection-v1 `
  -ConfigPath C:\OVA\private\collection.json
C:\OVA-Runtime\collection-venv-v1\Scripts\python.exe -B `
  .\windows\collection\tests\test_native_collection.py `
  --evidence C:\OVA-Acceptance\collection-native-unique.json
```

Preparation verifies wheels, creates a new venv, checks dependencies and derives code. It performs no service registration or customer scan. Revision03 actual Windows acceptance passed all six tests with no skips: synthetic sources and real process locks, concurrent publication/readers, restart journal reuse, redaction, Parquet, authenticated HTTP query/search, byte-verified export and native supervision. A held legacy reader released publication after 0.125 seconds; permanent sharing denial preserved the old bytes. The recorded 0.0-second post-death reacquisition is fixture clock resolution after the child wait, not a total kill-to-release guarantee. Preserve failing receipts as well as successful retests. POSIX preparation checks are explicitly not native Windows acceptance.

Windows may defer lock cleanup after abrupt process death. Acceptance permits reacquisition within five seconds, while exclusion remains immediate when the owner is alive. The implementation retains real kernel locks; there is no no-op production fallback. Immutable generation readers pin a verified regular `current.json` pointer, avoiding symbolic-link privileges.

File bytes are flushed before `MoveFileExW(REPLACE_EXISTING | WRITE_THROUGH)` publication. POSIX directory fsync is unavailable; Windows storage power-loss and actual reboot survival remain unqualified. Native supervision replaces Linux timers, but it does not reproduce systemd CPU/memory cgroups; apply a reviewed Windows resource policy if required.

For actual capture, configure approved native mirror roots and/or the narrow trusted-SSH scheduler path. Establish durable accounting/completion sources and source retention before the first scoped job. Test exact-byte coverage against an independent scheduler inventory. Synthetic success does not establish live Slurm capture, array/federation adapters or recovery of bytes deleted before capture. Preserve existing state/checkpoints on upgrade.

`service_wrapper.py` can render an external descriptor around the vetted WinSW binary. Rendering does not register a service. Account ACLs, service lifecycle and reboot commissioning remain

separate steps.

9. Configure one-node Auto Action control

The native adapter hosts scheduling, exclusive invocation and receipts on Windows. It invokes the original reviewed `actions/launch_job.sh` through the head's existing `fanout.sh` abstraction and a selected `clush`/`pdsh` provider. Windows does not fan out directly to GPU nodes or install the upstream watchdog.

Copy `windows/auto-action/site.example.json` to a private file. Set one exact node and test, reviewed remote root, launcher SHA-256, timeout, fanout provider and external state directory. Keep `mode: preview` and `execute_authorized: false` initially.

```
Set-Location C:\OVA\repository
& $Python windows\auto-action\controller.py plan --config C:\OVA\private\auto-action.json
& $Python windows\auto-action\controller.py cycle --config C:\OVA\private\auto-action.json
& $Python windows\auto-action\controller.py render-task `
  --config C:\OVA\private\auto-action.json `
  --python $Python `
  --output C:\OVA\private\auto-action-task.xml
```

Preview uses the original launcher's explain/test-selection path without submission. The generated task is disabled and unregistered. Review account/configuration before explicitly registering and enabling it during commissioned operations.

Execution requires the private authorization flag and invokes the real test path, which can drain the selected node on failure. Define GPU/node/runtime/privilege/cleanup scope first. The adapter does not enable fleet recovery or clear existing drains. SSH error, timeout or missing completion marker is recorded as uncertain; later cycles refuse to replay unresolved attempts because remote work may already exist. Inspect scheduler/results before resolving an attempt; do not delete state to hide uncertainty. A successful transport exit does not prove the GPU test passed.

10. Prepare Exemplar recipes and payloads

Native preparation validates the original schema and supported scale arithmetic, pins upstream identity and verifies actual local payload bytes. It separately validates remote Linux destination paths, rather than interpreting them as Windows filesystem paths.

```
Set-Location C:\OVA\repository
& $Python windows\exemplar\build_helper.py C:\OVA-Versions\exemplar-preparation-v1
& $Python C:\OVA-Versions\exemplar-preparation-v1\prepare.py validate-config C:\OVA\private\exemplar-site.json
& $Python C:\OVA-Versions\exemplar-preparation-v1\prepare.py manifest C:\OVA\payload C:\OVA\private\payload-
manifest.json
& $Python C:\OVA-Versions\exemplar-preparation-v1\prepare.py verify C:\OVA\payload C:\OVA\private\payload-
manifest.json
```

Select the exact supported GPU architecture, precision, recipe and scale; pin matching source/container/model assets. NGC/Hugging Face credentials are needed only for the selected protected assets and remain private file references. Windows helper success does not establish a workload's upstream support or installation closure.

Use the original component handoff for SSH-controlled Linux installation and Slurm/Enroot/Pyxis execution. Verify recipient hashes, executable modes and any symbolic links on Linux before launch. Scope the first GPU canary by node, partition/account, GPU count, duration and output destination. Keep GPU compute and bulk peer traffic inside the cluster. A scheduler exit code does not by itself establish performance qualification.

11. Connect the existing Windows agent and retain evidence

Direct coordination uses the configured Codex app-server endpoint and existing thread. Do not create GitLab issues or repository files as an agent-message transport. Code publication to GitLab is a separate activity. Use the endpoint, authentication mechanism and thread identity supplied through the approved session; do not guess or expose an app server publicly.

The inspected CLI 0.166.0 supports:

```
codex queue --remote <APP_SERVER_ENDPOINT> `
  --remote-auth-token-env <TOKEN_ENVIRONMENT_VARIABLE_NAME> `
  --thread <EXISTING_THREAD_UUID_OR_EXACT_NAME> `
  --message "Please run the agreed read-only OVA native Windows audit."
```

`--resume` separately attempts thread execution. Preserve the existing agent's ownership and installed state. Request source-only adapter exports with manifests, redacted runtime receipts, executable/service identities, exact upstream/derived hashes and per-tool placements. Do not copy private configuration, keys, state databases, inventories or logs wholesale into shared artifacts.

Each acceptance report must name host OS/build, interpreter/runtime version, source/adapter identity, exact check and result, cleanup outcome, excluded scope and evidence destination. Native process startup, isolated synthetic acceptance, service commissioning and customer end-to-end acceptance are distinct gates.

12. Operations, rollback and handover

Check VPN state, endpoint aliases, service identity, source freshness, disk/retention and collector labels before a session. Keep runtime configuration private and separate from immutable source. Back up collection state, Grafana configuration/credential database, telemetry data and required P2P metadata using the component's consistent backup procedure; prove restore into an isolated destination.

Stage updates into new version directories, verify input/output receipts and preserve existing state. Test one component/canary before expansion. Rollback code/configuration only through the component's supported path; database/schema downgrades may require an appropriate backup. Do not reset collector identity, job IDs or checkpoints to hide gaps.

Disconnecting VPN or closing a browser does not cancel existing jobs or stop remote services. Native foreground processes and optional wrappers have different lifetimes; record what remains running. Stop only owned processes with the supplied profile controls. Do not use blanket scheduler cancellation, drain clearing or unrelated service restarts as cleanup.

The installed-state handover records exact source/adapter hashes; Windows OS/build; real service/executable paths; native versus remote placement; listener/tunnel map; private configuration/credential locations without contents; persistent state/backup roots; collector/node scope; test and job IDs; acceptance gaps; owner; update and rollback instructions. Keep the P2P Linux-core exception, PythonCM execution guard and storage/lifecycle limits explicit.

13. Troubleshooting by boundary

Symptom	Discriminating check
Native import or wheel failure	Actual CPython 3.12 x64 interpreter and fully materialized Windows wheelhouse; Linux wheels and embedded 3.14 controls are different runtimes
Hash or adapter build mismatch	Exact manifest/base/export/patch identity; do not edit pins to conceal changed input
BCM reachable but app-server fails	Separate DNS/route/port/identity and VPN ACL for the second endpoint
Rack inventory works but workflow is rejected	Expected PythonCM fail-closed boundary; inventory does not qualify execution-session cookies or task semantics
P2P console loads but login/transfer fails	Real Manager tunnel/API, Linux core health and actual seed/peer path; mock proxy acceptance is narrower
Native Observability ready but cluster metrics absent	Explicit discovery, telemetry direction, approved ingress/tunnel, labels and source permissions
Collection lock or path failure	Match the tested native adapter revision, actual file-object API and canonical Windows path; preserve traces and real exclusion
Collection gaps after restart	Source retention, accounting/completion inventory, durable identity, size limits and reported gaps; no recovery promise for deleted bytes
Auto Action state is uncertain	Inspect the remote scheduler and completion evidence before any replay; retain the durable attempt record
Foreground test passes but service fails	Service account/profile ACL, explicit interpreter, wrapper variant, SSH identity, environment and actual restart/reboot behavior

14. Historical delivery and reference material

The original revision02 six-tool bundle was designed for native Windows staging/SSH/browser control with Linux backends. Its `control/agent_handoff.md` and tool handoffs remain versioned references, not the primary native profile installation instructions. Do not deploy heavy Linux backends or enable virtualization to make a native-only acceptance appear complete.

The historical Linux Observability dependency packs and image-based stack do not execute natively on Windows. The new Windows core has the explicit feature/datasource limits in section 7. Historical full-object-store P2P, Rack provisioning actions and selected NVIDIA workload payloads retain independent gates; synthetic core checks do not qualify those optional paths.

Read `windows/README.md`, each native profile's README/OPERATIONS document and exact dependency/validation manifest before execution. The original framework documentation still determines cluster-side recipe semantics, collection capture limits and thin-node prerequisites. Keep documentation and evidence tied to the actual installed revision.

Original distribution: NVFTP IREN_MKZ. Source publication: NVIDIA OVA Monorepo Windows.

Use executable files from the checkout. The PDF is an operator reference; replace placeholders with approved private values and inspect the exact candidate before running commands.